

SESSION 5 — Handling Missing Values in Pandas

Why Handling Missing Values is Important

In real-world datasets, it is very common to encounter missing values.

Examples:

Customer	Age	City
Aman	25	Delhi
Priya	NaN	Mumbai
Rahul	30	Delhi

Missing values can occur due to:

- Data entry errors
- Incomplete forms
- System failures
- Data collection issues

If missing values are not handled properly, they can cause:

- Incorrect analysis
- Errors in machine learning models
- Biased results

Therefore data cleaning is a critical step in data analysis.

Identifying Missing Values

Pandas represents missing values using **NaN**.

Function used:

`isnull()`

Syntax

`df.isnull()`

Purpose

Returns a boolean DataFrame where:

True → value is missing

False → value is present

```
import pandas as pd
import numpy as np

data = {
    "Customer": ["Aman", "Priya", "Rahul", "Sneha"],
    "Age": [25, np.nan, 30, np.nan],
    "City": ["Delhi", "Mumbai", "Delhi", "Bangalore"]
}

df = pd.DataFrame(data)

df
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	NaN	Mumbai
2	Rahul	30.0	Delhi
3	Sneha	NaN	Bangalore

```
df.isnull()
```

	Customer	Age	City
0	False	False	False
1	False	True	False
2	False	False	False
3	False	True	False

Counting Missing Values

To count missing values per column we use:

```
df.isnull().sum()
```

```
df.isnull().sum()
```

	0
Customer	0
Age	2
City	0

```
dtype: int64
```

Removing Missing Values

Function:

```
dropna()
```

Syntax

```
df.dropna(axis=0, how='any')
```

Parameters

axis

0 → remove rows

1 → remove columns

how

'any' → drop if any value is missing

'all' → drop if all values are missing

```
df.dropna()
```

	Customer	Age	City
0	Aman	25.0	Delhi
2	Rahul	30.0	Delhi

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Alice", "Bob", "Charlie", "David", None],
    "Age": [25, None, 30, None, None],
    "City": ["New York", "London", None, None, None]
}

df = pd.DataFrame(data)
```

```
print(df)
```

	Name	Age	City
0	Alice	25.0	New York
1	Bob	NaN	London
2	Charlie	30.0	None
3	David	NaN	None
4	None	NaN	None

```
df.dropna(axis=0)
```

	Name	Age	City
0	Alice	25.0	New York

```
df.dropna(axis=0, how='any')
```

	Name	Age	City
0	Alice	25.0	New York

Rows removed:

Row 1 → Age missing

Row 2 → City missing

Row 3 → Age & City missing

Row 4 → All missing

```
df.dropna(axis=0, how='all')
```

	Name	Age	City
0	Alice	25.0	New York
1	Bob	NaN	London
2	Charlie	30.0	None
3	David	NaN	None

```
df.dropna(axis=1, how='any')
```

0
1
2
3
4

✓ Filling Missing Values

Instead of removing rows, we may replace missing values.

Function:

`fillna()`

Syntax

`df.fillna(value)`

Common strategies:

- Replace with mean
- Replace with median

- Replace with constant value

```
df["Age"].fillna(df["Age"].mean())
```

	Age
0	25.0
1	27.5
2	30.0
3	27.5

dtype: float64

```
# not my code
# FILLING MISSING VALUES:--
import pandas as pd
import numpy as np

data={'customer':['aman','priya','rahul','sneha'],
      'age':[25,np.nan,30,np.nan],
      'city':['delhi','mumbai',np.nan,'banglore']}

df=pd.DataFrame(data)
print(df)
b='dropped'
#df['age'].fillna(b)
c = df['age'].mean()
a = "mehsana"
# df[['age','city']].fillna({df['age'].mean(),df['city'] : a}) # .fillna(value) you can put any values with it.
df.fillna({
    "age" : c,
    "city" : "mehsana",
    # "City" : "Ahmedabad"
})
```

	customer	age	city
0	aman	25.0	delhi
1	priya	NaN	mumbai
2	rahul	30.0	NaN
3	sneha	NaN	banglore

	customer	age	city
0	aman	25.0	delhi
1	priya	27.5	mumbai
2	rahul	30.0	mehsana
3	sneha	27.5	banglore

```
value = 44
# df.fillna(value)
# df["Age"].fillna(value)
df.fillna({
    "Name" : "Aman",
    "Age" : value,
    "City" : "Ahmedabad"
})
```

	Name	Age	City
0	Alice	25.0	New York
1	Bob	44.0	London
2	Charlie	30.0	Ahmedabad
3	David	44.0	Ahmedabad
4	Aman	44.0	Ahmedabad

Replacing Missing Values with Median

Median is often preferred when data contains outliers.

```
df["Age"].fillna(df["Age"].median())
```

```

      Age
0  25.0
1  27.5
2  30.0
3  27.5

dtype: float64

```

Case Study — Cleaning Missing Ages

Goal:

Replace missing ages using average age.

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
df
```

```

      Customer  Age  City
0      Aman  25.0  Delhi
1      Priya  27.5  Mumbai
2      Rahul  30.0  Delhi
3      Sneha  27.5  Bangalore

```

Handling Missing Values in Categorical Columns

Pandas supports multiple data types in a DataFrame, such as:

- Numerical → Age, Salary
- Categorical → City, Gender, Product
- Boolean → True/False
- Datetime → Dates

Missing values (NaN) can exist in **any type of column**, not just numerical.

Important Clarification

- dropna() works on all columns
- fillna() works on all columns

BUT

The way we **fill missing values depends on the data type**

Handling Numerical vs Categorical Data

Numerical Columns:

- Replace with mean
- Replace with median

Categorical Columns:

- Replace with mode (most frequent value)
- Replace with constant like "Unknown"

Why Mean Does Not Work for Categorical Data

Example:

City column:

Delhi, Mumbai, NaN

We cannot calculate mean of "Delhi" and "Mumbai"

So we use:

- Mode (most frequent value)
- Constant value like "Unknown"

```
import pandas as pd
import numpy as np

data = {
    "Customer": ["Aman", "Priya", "Rahul", "Sneha"],
    "Age": [25, np.nan, 30, np.nan],          # Numerical
    "City": ["Delhi", np.nan, "Delhi", "Mumbai"] # Categorical
}

df = pd.DataFrame(data)

print("Original DataFrame:\n")
print(df)
```

Original DataFrame:

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	NaN	NaN
2	Rahul	30.0	Delhi
3	Sneha	NaN	Mumbai

1. Filling Numerical Column (Age)

Use mean or median

```
df["Age"] = df["Age"].fillna(df["Age"].mean())

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	NaN
2	Rahul	30.0	Delhi
3	Sneha	27.5	Mumbai

2. Filling Categorical Column Using Mode

Mode = most frequent value

```
# FILLING MISSING VALUES:--
import pandas as pd
import numpy as np

data={'customer':['aman','priya','rahul','sneha'],
      'age':[25,np.nan,30,np.nan],
      'city':['delhi','delhi',np.nan,'banglore']}

df=pd.DataFrame(data)
#print(df)
b='dropped'
```

```
#df['age'].fillna(b)
#df['age'].fillna(df['age'].mean()) # .fillna(value) you can put any values with it.
df['city']=df['city'].fillna(df['city'].mode()[0])
```

```
df["City"] = df["City"].fillna(df["City"].mode()[0])

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	Delhi
2	Rahul	30.0	Delhi
3	Sneha	27.5	Mumbai

3. Filling Categorical Column Using Constant

Instead of mode, we can use a constant value.

Example:

"Unknown"

```
df = pd.DataFrame(data)

df["City"] = df["City"].fillna("Unknown")

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	NaN	Unknown
2	Rahul	30.0	Delhi
3	Sneha	NaN	Mumbai

4. Using fillna() for Multiple Columns

We can handle both numerical and categorical columns together.

Example:

- Age → replace with mean
- City → replace with "Unknown"

```
df = pd.DataFrame(data)

df = df.fillna({
    "Age": df["Age"].mean(),
    "City": "Unknown"
})

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	Unknown
2	Rahul	30.0	Delhi
3	Sneha	27.5	Mumbai

SESSION 6 — Handling Duplicates and Outliers

Two common problems in datasets are:

- Duplicate records
- Outliers (extreme values)

Both can distort analysis results.

Handling Duplicate Rows

Function:

`drop_duplicates()`

Syntax

`df.drop_duplicates()`

Purpose

Removes duplicate rows from the dataset.

Parameters

subset

Specify column(s) to check duplicates

keep

'first' → keep first occurrence

'last' → keep last occurrence

False → remove all duplicates

```
data = {  
    "Customer": ["Aman", "Priya", "Aman", "Rahul"],  
    "Sales": [500, 700, 500, 900]  
}
```

```
df = pd.DataFrame(data)
```

df

	Customer	Sales
0	Aman	500
1	Priya	700
2	Aman	500
3	Rahul	900

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.drop_duplicates()
```

	Customer	Sales
0	Aman	500
1	Priya	700
3	Rahul	900

What are Outliers?

Outliers are extreme values that differ significantly from other observations.

Example:

[10, 12, 11, 13, 500]

Value **500** is an outlier.

Outliers can occur due to:

- measurement errors
- data entry errors
- unusual events

Detecting Outliers using IQR Method

IQR = Interquartile Range

Formula:

$$\text{IQR} = Q3 - Q1$$

$$\text{Lower Bound} = Q1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q3 + 1.5 \times \text{IQR}$$

```
sales = pd.Series([100,120,130,110,115,500])

Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = sales[(sales < lower) | (sales > upper)]

outliers
```

```
0
5 500

dtype: int64
```

Z-Score Concept

Z-score measures how far a value is from the mean.

Formula:

$$Z = (X - \text{Mean}) / \text{Standard Deviation}$$

Values with $|Z| > 3$ are often considered outliers.

```
mean = sales.mean()
std = sales.std()

z_scores = (sales - mean) / std

z_scores
```

SESSION 7 — Column Operations & Data Transformations

In data analysis we often need to:

- Create new columns
- Modify existing columns
- Apply functions to columns

Pandas provides several methods to perform transformations.

Adding New Columns

A new column can be created using assignment.

Example:

$$\text{Revenue} = \text{Price} \times \text{Quantity}$$

```
import pandas as pd

data = {
    "Product": ["Pen", "Notebook", "Marker"],
    "Price": [10, 50, 20],
    "Qty": [100, 40, 60]
}

df = pd.DataFrame(data)

df
```

	Product	Price	Qty	
0	Pen	10	100	
1	Notebook	50	40	
2	Marker	20	60	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df["Revenue"] = df["Price"] * df["Qty"]

df
```

	Product	Price	Qty	Revenue	
0	Pen	10	100	1000	
1	Notebook	50	40	2000	
2	Marker	20	60	1200	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Using apply()

apply() allows applying a function to columns.

Syntax

```
df["column"].apply(function)
```

Commonly used for custom transformations.

```
df["Price_with_tax"] = df["Price"].apply(lambda x: x * 1.18)

df
```

	Product	Price	Qty	Revenue	Price_with_tax	
0	Pen	10	100	1000	11.8	
1	Notebook	50	40	2000	59.0	
2	Marker	20	60	1200	23.6	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Using lambda Functions

Lambda functions are small anonymous functions.

Syntax:

lambda argument : expression

▼ Using applymap()

applymap() applies a function to **every element of the DataFrame**.

Syntax

df.applymap(function)

Note:

Used only for DataFrames (not Series).

```
df_numeric = df[["Price", "Qty"]]

# df_numeric.applymap(lambda x: x * 2)
df_numeric.map(lambda x: x * 2)
```

	Price	Qty
0	20	200
1	100	80
2	40	120

▼ Case Study — Revenue Calculation

Create a new column:

Revenue = Price × Quantity

Then calculate total revenue.

```
df["Revenue"] = df["Price"] * df["Qty"]

total_revenue = df["Revenue"].sum()

total_revenue

np.int64(4200)
```

▼ Pandas Practice Set (Sessions 1–7)

This set covers:

- DataFrame creation
- Selection (loc, iloc)
- Filtering
- Sorting
- Missing values
- Duplicates
- Outliers
- Column transformations

Question 1

Create a DataFrame with columns:

Name, Age, City, Salary

Tasks:

- Display only the Age column
- Display Name and Salary together

Question 2

Using the same dataset:

- Select the 3rd row using **iloc**
 - Select the Salary of the 2nd row using **loc**
-

Question 3

Filter the dataset:

- Show rows where Salary > 50000
 - Show rows where City is "Delhi"
-

Question 4

Apply multiple conditions:

- Salary > 40000 AND City = "Mumbai"
-

Question 5

Sort the dataset:

- Sort by Salary in ascending order
 - Sort by Salary in descending order
-

Question 6

After sorting:

- Reset the index
 - Remove the old index
-

Question 7

Create a dataset with missing values in Age.

Tasks:

- Detect missing values using `isnull()`
 - Count missing values per column
-

Question 8

Using the same dataset:

- Drop rows with missing values
 - Drop columns with missing values
-

Question 9

Fill missing values:

- Replace missing Age with mean
 - Replace missing City with "Unknown"
-

Question 10

Create a dataset with duplicate rows.

Tasks:

- Identify duplicates
 - Remove duplicate rows
-

Question 11

Create a Sales column:

[100, 120, 110, 115, 500]

Tasks:

- Detect outliers using IQR method
-

Question 12

Using the same sales data:

- Calculate Z-score for each value
 - Identify values where Z-score > 2
-

Question 13

Create a dataset:

Product, Price, Quantity

Tasks:

- Create new column Revenue = Price * Quantity
 - Calculate total revenue
-

Question 14

Use apply() and lambda:

- Increase all prices by 10%
 - Create a new column Discounted Price = Price - 10%
-

Final Integrated Question (Important)

Question 15 — Mini Data Cleaning & Analysis Task

Create a dataset:

Customer, Age, City, Price, Quantity

Include:

- Missing values in Age and City
- Duplicate rows

Tasks:

1 Clean missing values

- Replace Age with mean
- Replace City with "Unknown"

2 Remove duplicate rows

3 Create new column:

Revenue = Price * Quantity

4 Filter rows where Revenue > 500

5 Sort the dataset by Revenue in descending order

6 Reset the index

7 Identify any outliers in Revenue using IQR method

End of Practice Set

Start coding or [generate](#) with AI.

